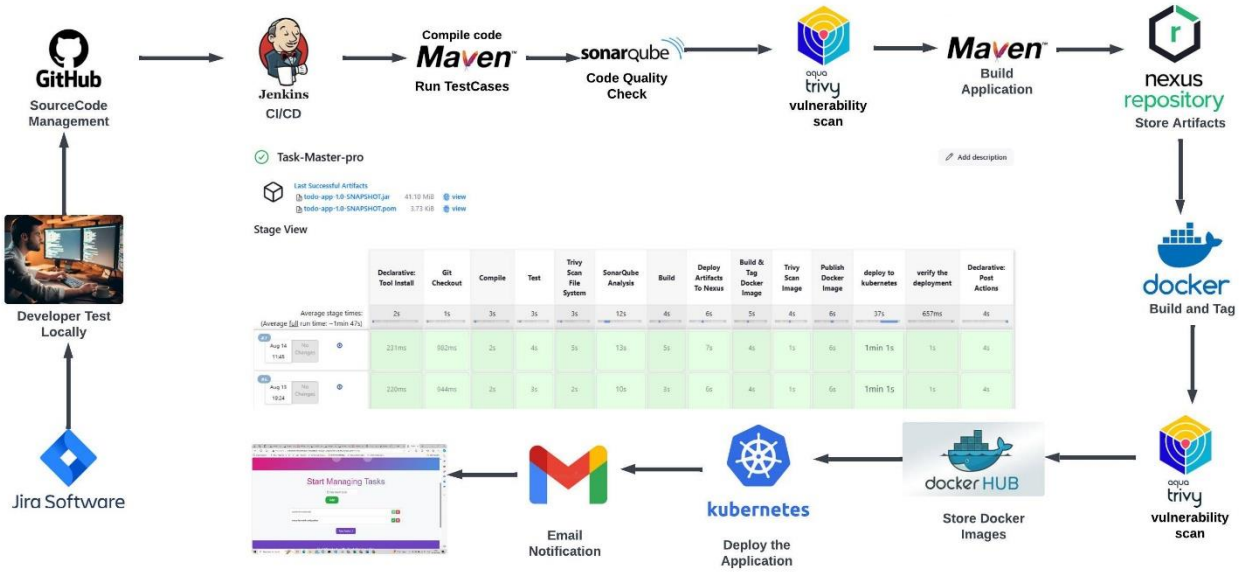
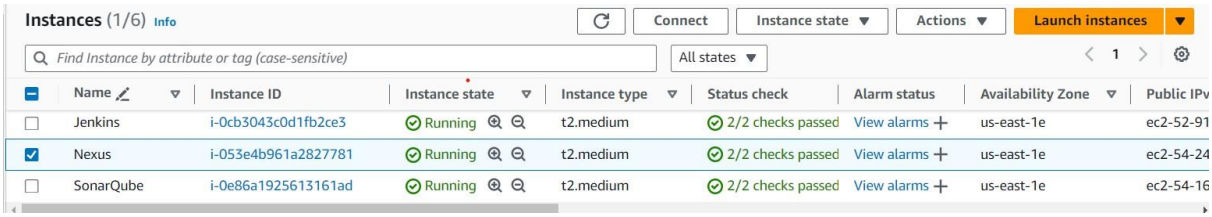
The Ultimate End-End CI/CD Project



Phase-1

Create 3 EC2 Instances with 20GB RAM and choose t2.medium



Instances (1/6) Info

Connect | Instance state | Actions | Launch instances

Find Instance by attribute or tag (case-sensitive) | All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv
Jenkins	i-0cb3043c0d1fb2ce3	Running	t2.medium	2/2 checks passed	View alarms +	us-east-1e	ec2-52-91
Nexus	i-053e4b961a2827781	Running	t2.medium	2/2 checks passed	View alarms +	us-east-1e	ec2-54-24
SonarQube	i-0e86a1925613161ad	Running	t2.medium	2/2 checks passed	View alarms +	us-east-1e	ec2-54-16

Install Docker on All 3 VMs

Step-by-Step Installation

1. Install prerequisite packages:

sudo apt-get update

sudo apt-get install ca-certificates curl

2. Download and add Docker's official GPG key:

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

3. Add Docker repository to Apt sources:

echo "deb [arch=$(dpkg --print-architecture) signed by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

4. Update package index:

sudo apt-get update

5. Install Docker packages:

sudo apt-get install docker-ce docker-ce-cli containerd.io -y

6. Grant permission to Docker socket (optional, for convenience):

sudo chmod 666 /var/run/docker.sock

By following these steps, you should have successfully installed Docker on your Ubuntu system. You can now start using Docker to containerize and manage your applications.

Follow this official document if you find any errors:

Link: [Install Docker Engine on Ubuntu | Docker Docs]

Setting Up Jenkins on Ubuntu

Step-by-Step Installation

1. Update the system:

sudo apt-get update

sudo apt-get upgrade -y

2. Install Java (Jenkins requires Java):

sudo apt install -y fontconfig openjdk-17-jre

3. Add Jenkins repository key:

sudo wget -O /usr/share/keyrings/jenkins-keyring.asc

https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key

4. Add Jenkins repository:

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]

https://pkg.jenkins.io/debian-stable binary/" | sudo tee

/etc/apt/sources.list.d/jenkins.list > /dev/null

5. Update the package index:

sudo apt-get update

6. Install Jenkins:

sudo apt-get install -y jenkins

7. Start and enable Jenkins:

sudo systemctl start jenkins

sudo systemctl enable jenkins

8. Access Jenkins:

- Open a web browser and go to http://your_server_ip_or_domain:8080.
- You will see a page asking for the initial admin password. Retrieve it using:
- sudo cat /var/lib/jenkins/secrets/initialAdminPassword
- Enter the password, install suggested plugins, and create your first admin user.

or follow this official document

link: https://www.jenkins.io/doc/book/installing/linux/#debianubuntu

Installing Trivy on Jenkins Server

Step-by-Step Installation

1. Install prerequisite packages:

sudo apt-get install wget apt-transport-https gnupg lsb-release

2. Add Trivy repository key:

wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key |

sudo apt-key add -

3. Add Trivy repository to sources:

echo deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main | sudo tee -a /etc/apt/sources.list.d/trivy.list

4. Update package index:

sudo apt-get update

5. Install Trivy:

sudo apt-get install trivy

or follow this official document

link: https://aquasecurity.github.io/trivy/v0.18.3/installation/

SonarQube Setup:

Ssh into sonarqube ec2 instance

docker run -d --name sonar -p 9000:9000 sonarqube:lts-comminity

access using <publicip:9000>

username: admin

password:admin

Nexus Setup:

Ssh into nexus ec2 instance

docker run -d --name nexus -p 8081:8081 sonatype/nexus3

access using <publicip:8081>

sign to nexus using the password, the password is stored in '**/nexus-data/admin.password**'

enter into the container using

docker exec -it <container-ID> /bin/bash

then run

cat **/nexus-data/admin.password**

you will get password

```
ubuntu@ip-172-31-59-116:~$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED
AMES
5559dd04aad5   sonatype/nexus3   "/opt/sonatype/nexus…"   About a minute ago
exus
ubuntu@ip-172-31-59-116:~$ docker exec -it 5559 /bin/bash
bash-4.4$ cat /nexus-data/admin.password
66db8137-b229-4682-a789-10655502bd3bbash-4.4$
```

Username: admin

Password: 66db8137-b229-4682-a789-10655502bd3b ###Replace your password

EKS-Setup:

First Create a user in AWS IAM with any name

Attach Policies to the newly created user

below policies

AmazonEC2FullAccess

AmazonEKS_CNI_Policy

AmazonEKSClusterPolicy

AmazonEKSWorkerNodePolicy

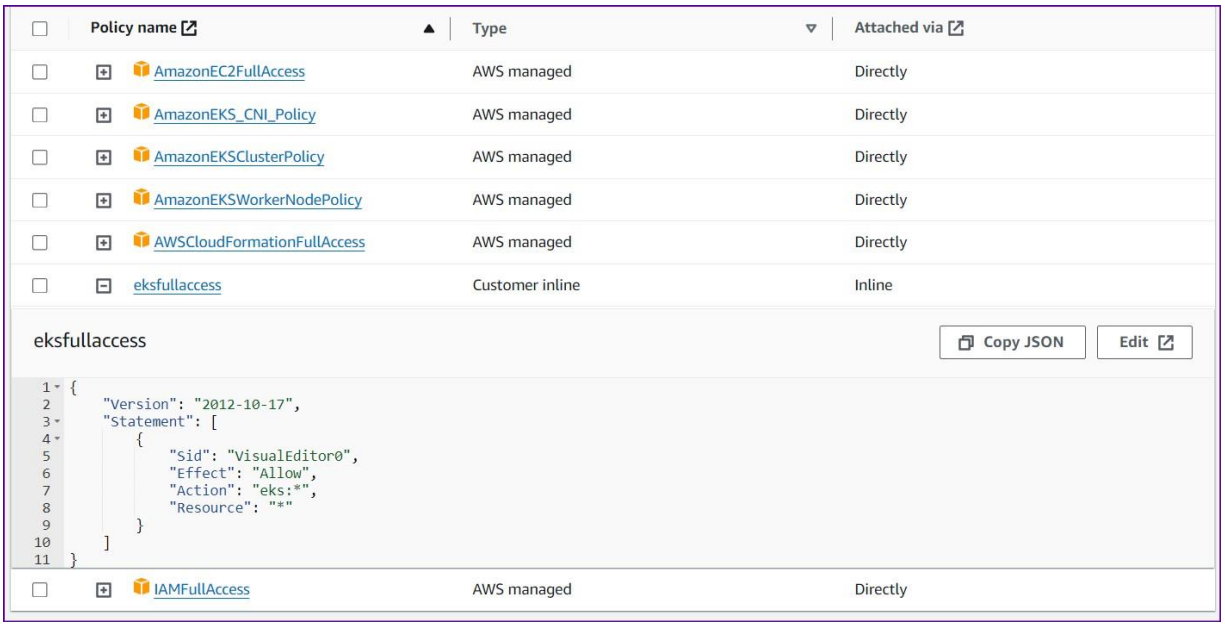
AWSCloudFormationFullAccess

IAMFullAccess

One more policy we need to create with content as below

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": "eks:*",
            "Resource": "*"
        }
    ]
}
```

Attach this policy to your user as well



Policy name	Type	Attached via
AmazonEC2FullAccess	AWS managed	Directly
AmazonEKS_CNI_Policy	AWS managed	Directly
AmazonEKSClusterPolicy	AWS managed	Directly
AmazonEKSWorkerNodePolicy	AWS managed	Directly
AWSCloudFormationFullAccess	AWS managed	Directly
eksfullaccess	Customer inline	Inline

eksfullaccess

Copy JSON Edit

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": "eks:*",
            "Resource": "*"
        }
    ]
}
```

IAMFullAccess	AWS managed	Directly

AWSCLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
sudo apt install unzip
unzip awscliv2.zip
sudo ./aws/install
aws configure
```

KUBECTL

```
curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-
01-05/bin/linux/amd64/kubectl
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin
kubectl version --short --client
```

EKSCTL

```
curl --silent --location
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(una
me -s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin
eksctl version
```

Create EKS CLUSTER

```
eksctl create cluster --name=my-eks7 \
                      --region=ap-south-1 \
                      --zones= ap-south-1a, ap-south-1b \
                      --version=1.30 \
                      --without-nodegroup
```

```
eksctl utils associate-iam-oidc-provider \
    --region us-east-1 \
    --cluster my-eks2 \
    --approve

eksctl create nodegroup --cluster=my-eks7 \
                        --region= ap-south-1\
                        --name=node2 \
                        --node-type=t3.medium \
                        --nodes=3 \
                        --nodes-min=2 \
                        --nodes-max=4 \
                        --node-volume-size=20 \
                        --ssh-access \
                        --ssh-public-key=panduaws \
                        --managed \
                        --asg-access \
                        --external-dns-access \
                        --full-ecr-access \
                        --appmesh-access \
                        --alb-ingress-access
```

Note: `--ssh-public-key=panduaws` → Give pem file name in AWS

- Open INBOUND TRAFFIC IN ADDITIONAL Security Group

Phase-2

Close the repository and create your own repository and push those into your github repository

1. clone the repo:

git clone https://github.com/Madeep9347/Task-Master-Pro.git

change the remote repo

git remote set-url origin https://github.com/Madeep9347/Task-Master-Pro.git

→ replace with your github repo

git remote add new-origin https://github.com/Madeep9347/Task-Master-Pro.git

→replace with your github repo

2. **Initialize Git Repository**

git init

3. **Add Files to Git:**

Stage all files for the first commit:

git add .

4. **Commit Files:**

Commit the staged files with a commit message:

git commit -m "Initial commit"

5. Push to GitHub:

Push the local repository to GitHub:

git push -u origin main

Jenkins Setup Complete:
Install Plugins in Jenkins

1. Eclipse Temurin installer → for jdk
2. Sonarqube scanner
3. Docker
4. Docker pipeline
5. Kubernetes
6. Kubernetes cli
7. Kubernetes credentials
8. Kubernetes clint api
9. Config file provider → for Nexus
10. Maven integration
11. Pipeline maven integration

Now we installed the tools and Now we need to configure them

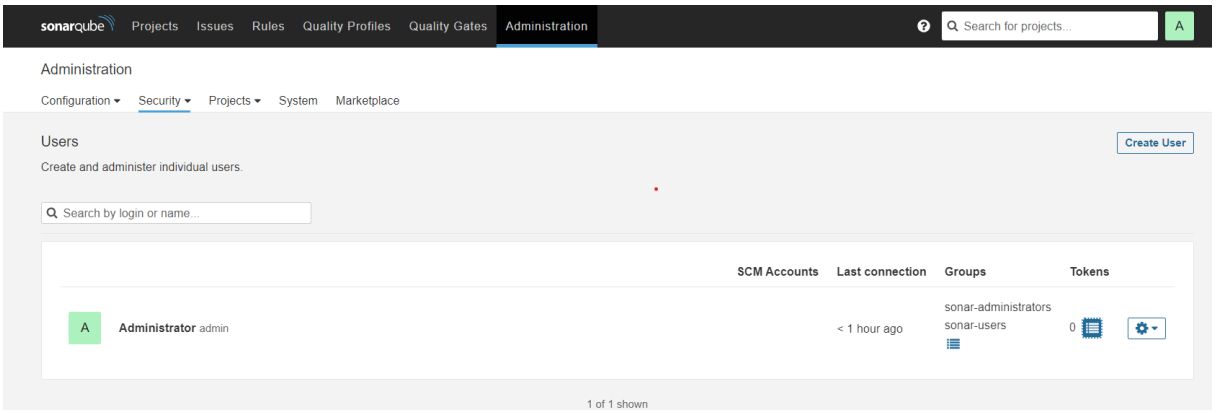
Go to → manage Jenkins→Tools→

1. Jdk→ name= jdk17 , install automatically from adoptium.net, version= jdk17 latest
2. Sonarqube scanner → name=sonar-scanner, Install automatically
3. Maven → name= maven3, version= 3.6.3
4. Docker→ name=docker, install automatically from docker.com

Now configure the sonarqube server in Jenkins

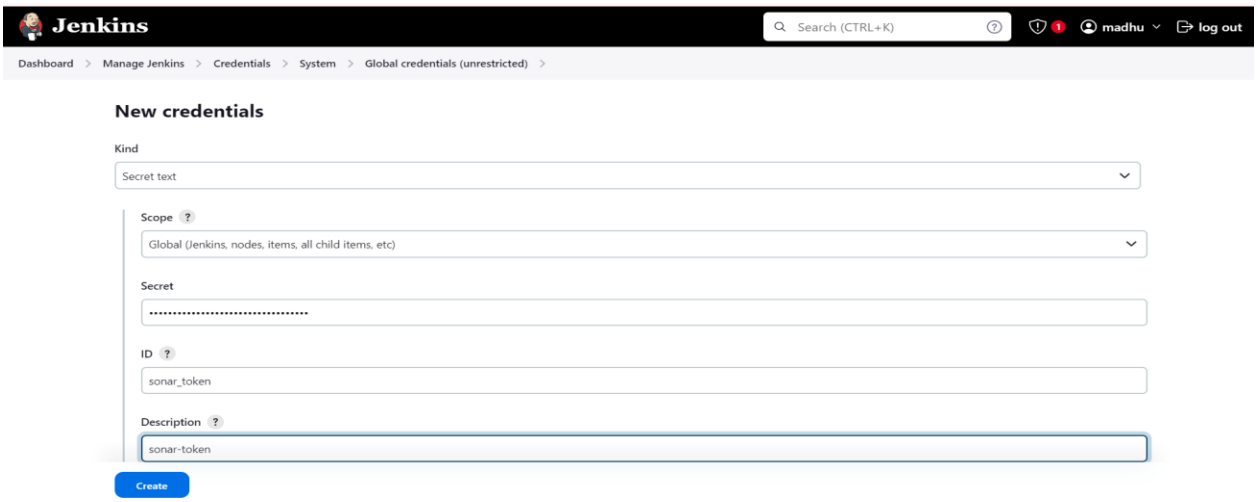
Firstly generate the token in sonarqube

Goto → Administaration→ security→ users→update token→ name= sonartoken and generate



add the token in jenkins

goto→ manage jenkins→credentials→ global→ kind= secret text→secret=<your-token>id=sonar-token, description=sonar=token



Go to → manage Jenkins→ system→sonarqube server→name=sonar, url=http://publicip:9000, token=sonar-token

Sonarqube scanner→ This is the tool that actually scans your code and sends the results to the SonarQube server.

Sonarqube server→ Displays analysis results.

Nexus Configuration:

Update your pom.xml file with your nexus repositories

```
</dependency>
        </dependencies>

        <distributionManagement>
        <repository>
            <id>maven-releases</id>
            <url>http://54.242.176.54:8081/repository/maven-releases/</url>
        </repository>
        <snapshotRepository>
            <id>maven-snapshots</id>
            <url>http://54.242.176.54:8081/repository/maven-snapshots/</url>
        </snapshotRepository>
    </distributionManagement>
```

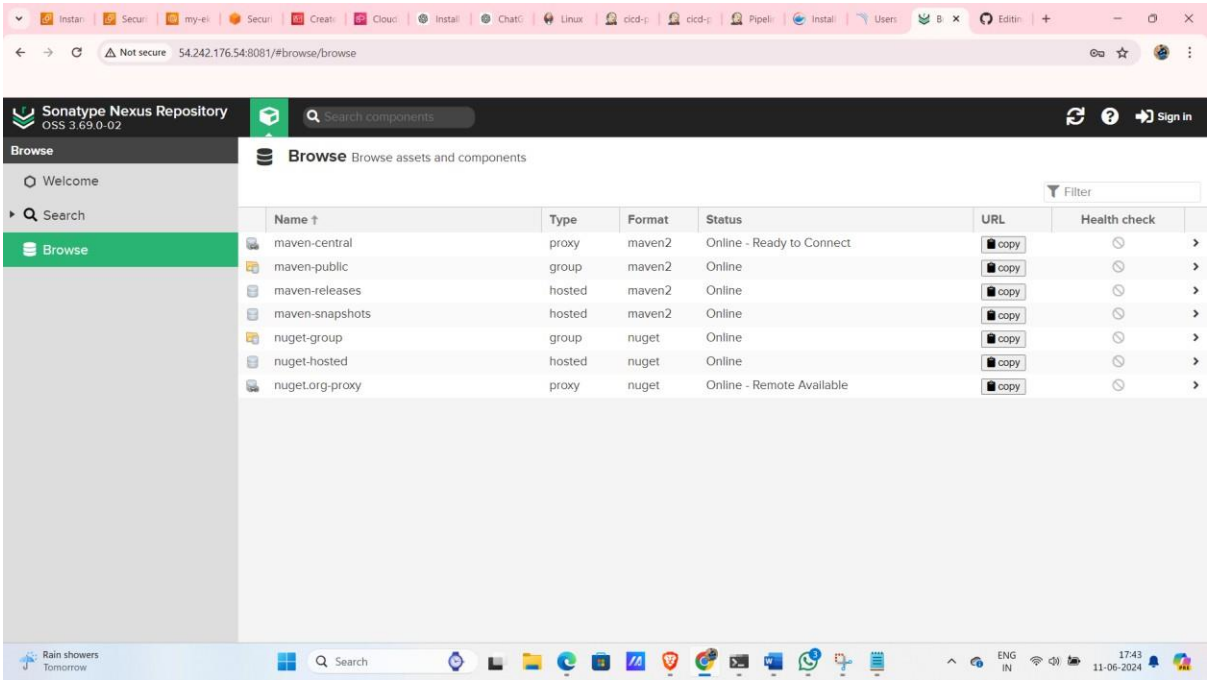
Copy the maven-releases URL , maven-snapshots URL and update in the pom.xml file

“”

<url>http://54.242.176.54:8081/repository/maven-releases/</url>

<url>http://54.242.176.54:8081/repository/maven-snapshots/</url>

“”

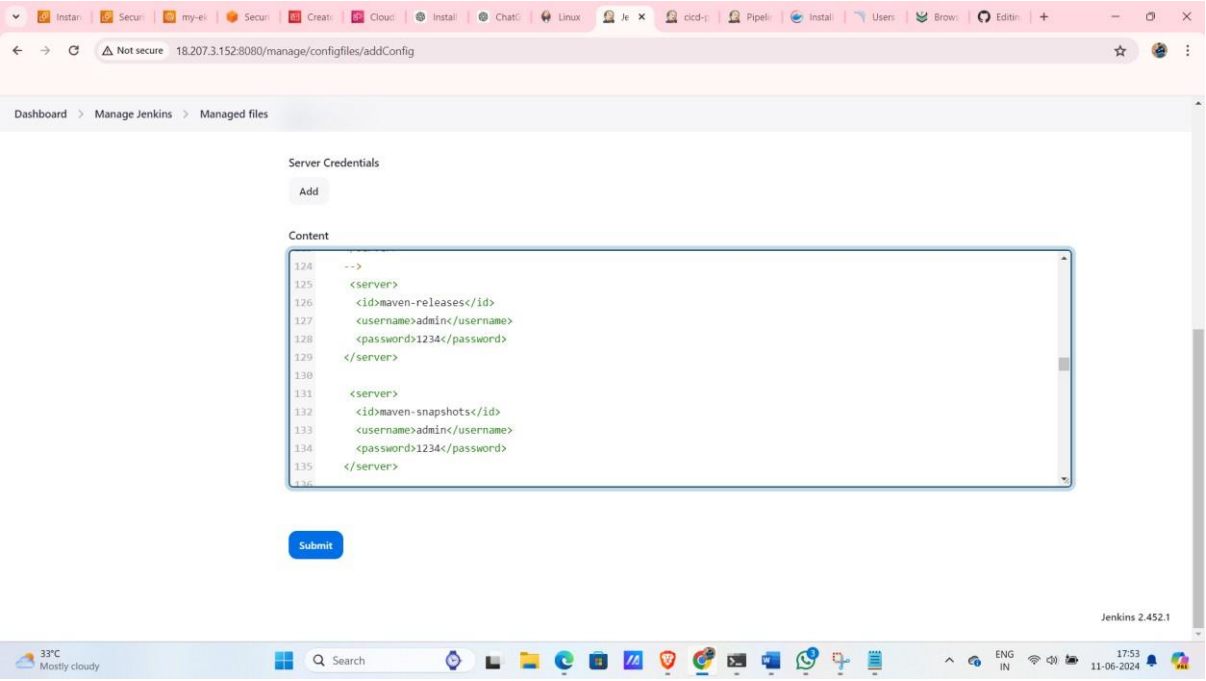


Nexus authentication with Jenkins:

Go to→ manage Jenkins→manage files→add new config→ select global mavensettings.xml, id=maven-setting → click on next

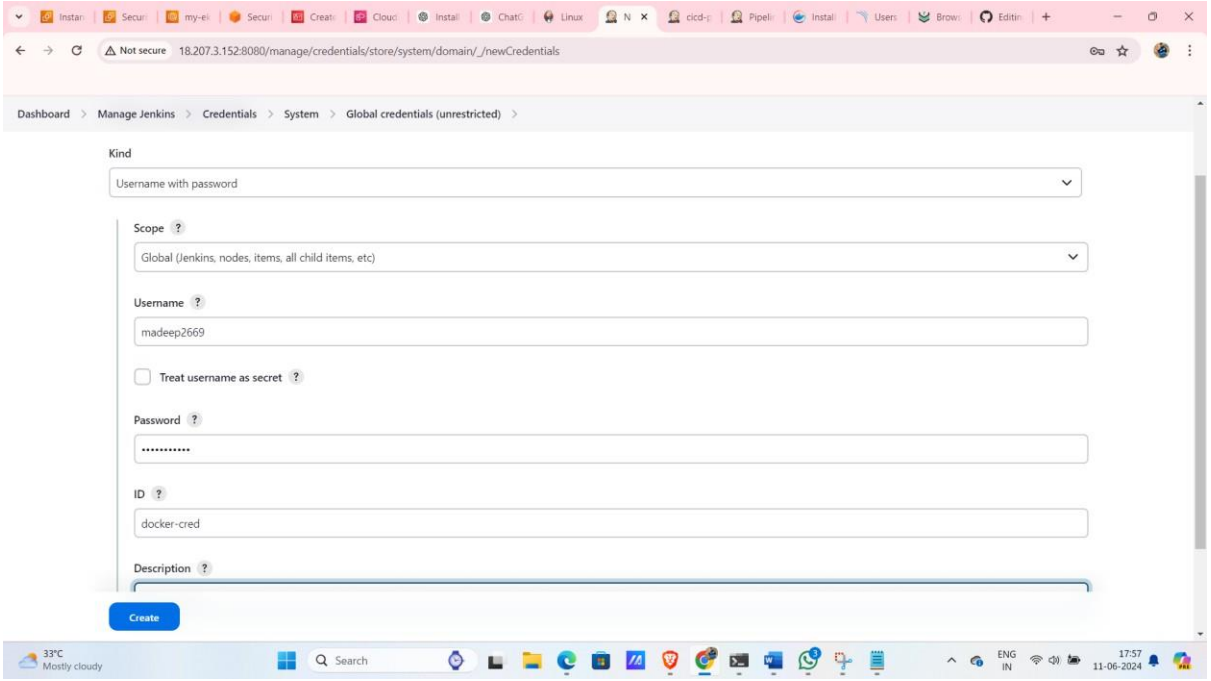
Go to content

Add the servers with name, username and password



Add DockerHub credentials in Jenkins:

Goto→ manage Jenkins→credentials→kind=username and password



Create Service Account, Role & Assign that role, And create a secret for Service Account and generate a Token in Jenkins server

Creating Service Account

First create the namespace using

Kubectl create namespace webapps

```
apiVersion:  v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: webapps
```

Create Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: app-role
  namespace: webapps
rules:
  - apiGroups:
```

```
        - ""
        - apps
        - autoscaling
        - batch
        - extensions
        - policy
        - rbac.authorization.k8s.io
    resources:
      - pods
      - secrets
      - componentstatuses
      - configmaps
      - daemonsets
      - deployments
      - events
      - endpoints
      - horizontalpodautoscalers
      - ingress
      - jobs
      - limitranges
      - namespaces
      - nodes
      - pods
      - persistentvolumes
      - persistentvolumeclaims
      - resourcequotas
      - replicasets
      - replicationcontrollers
      - serviceaccounts
      - services
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
```

Bind the role to service account

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-rolebinding
  namespace: webapps
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: app-role
subjects:
- namespace: webapps
  kind: ServiceAccount
  name: jenkins
```

Generate token using service account in the namespace:

```yaml
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: mysecretname
  annotations:
    kubernetes.io/service-account.name: myserviceaccount
```

kubectl -n webapps describe secret mysecretname

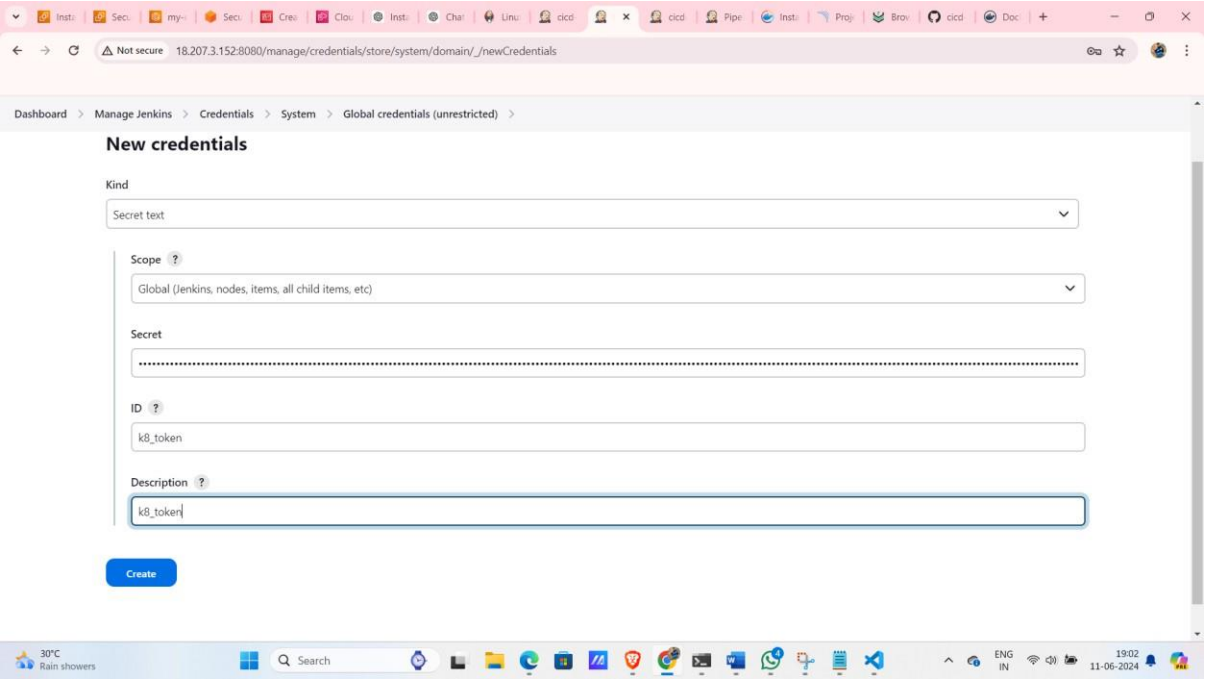
```
ubuntu@ip-172-31-54-141:~$ kubectl create namespace webapps
namespace/webapps created
ubuntu@ip-172-31-54-141:~$ vim service_account.yaml
ubuntu@ip-172-31-54-141:~$ kubectl apply -f service_account.yaml
serviceaccount/jenkins created
ubuntu@ip-172-31-54-141:~$ vim role.yaml
ubuntu@ip-172-31-54-141:~$ kubectl apply -f role.yaml
role.rbac.authorization.k8s.io/app-role created
ubuntu@ip-172-31-54-141:~$ vim role_bind.yaml
ubuntu@ip-172-31-54-141:~$ kubectl apply -f role_bind.yaml
rolebinding.rbac.authorization.k8s.io/app-rolebinding created
ubuntu@ip-172-31-54-141:~$ vim secret.yaml
ubuntu@ip-172-31-54-141:~$ kubectl apply -f se
secret.yaml          service_account.yaml
ubuntu@ip-172-31-54-141:~$ kubectl apply -f se
secret.yaml          service_account.yaml
ubuntu@ip-172-31-54-141:~$ kubectl apply -f se
secret.yaml          service_account.yaml
ubuntu@ip-172-31-54-141:~$ kubectl apply -f secret.yaml -n webapps
secret/mysecretname created
ubuntu@ip-172-31-54-141:~$ kubectl -n webapps describe secret mysecretname
Name:         mysecretname
Namespace:    webapps
Labels:       <none>
Annotations:  kubernetes.io/service-account.name: jenkins
              kubernetes.io/service-account.uid: 345fc6d1-ff35-47bb-a3d3-cf85fec3a0d7

Type:  kubernetes.io/service-account-token

Data
====
namespace:  7 bytes
token:      eyJhbGciOiJSUzI1NiIsImtpZCI6ImRINUUzZHoxX1VpU1VONkpGQmljSEFPckt4bGpIeDhmaDVxdV9BbHYzb3cifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZ
XJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJ3ZWJhcHBzIiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZWNyZXQubmFtZSI6Im15c2VjcmV0bmFtZSIsImt1YmVybmV0ZXMua
W8vc2VydmljZWFjY291bnQvc2VydmljZS1hY2NvdW50Lm5hbWUiOiJqZW5raW5zIiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQudWlkIjoiMzQ1ZmM2ZDEtZmYzNS00N
2JiLWEzZDMtY2Y4NWZlYzNhMGQ3Iiwic3ViIjoic3lzdGVtOnNlcnZpY2VhY2NvdW50OndlYmFwcHM6amVua2lucyJ9.s04GDXjsbB7P5YrWGSpNSNOv4kBiYLDuiVivaFUVtODYsge8v2L0TgdNdyK0Beur
xKLjECplBtruixRkvtJLY4zR9fWNv5M4P2TS0jsGafBXSzYIsDeDcmwOlR3YiP9s2f-KODYkjcHcTtLKaWyFGNRqUoOZdneRhxkndkL-S-gMHQ-zzd7ubyoU3w0HH3u2zW8NQae-3EOCVki-tpwhgauLU_Id
ALVA0cLP2gj2tMnba8XpLfFNA1hemQBt0cGha5PPAvjrJ27mlZ-LYe9ElvIglWsjkj-wvnGdv1q6GHgm7EQgALhSKJnBIMFWY8yhfAHQcsXjyLooOqz4wWq5lA
ca.crt:     1107 bytes
ubuntu@ip-172-31-54-141:~$
```

Add this token in Jenkins server

Goto→ manage Jenkins→ credentials→global→kind= secret text

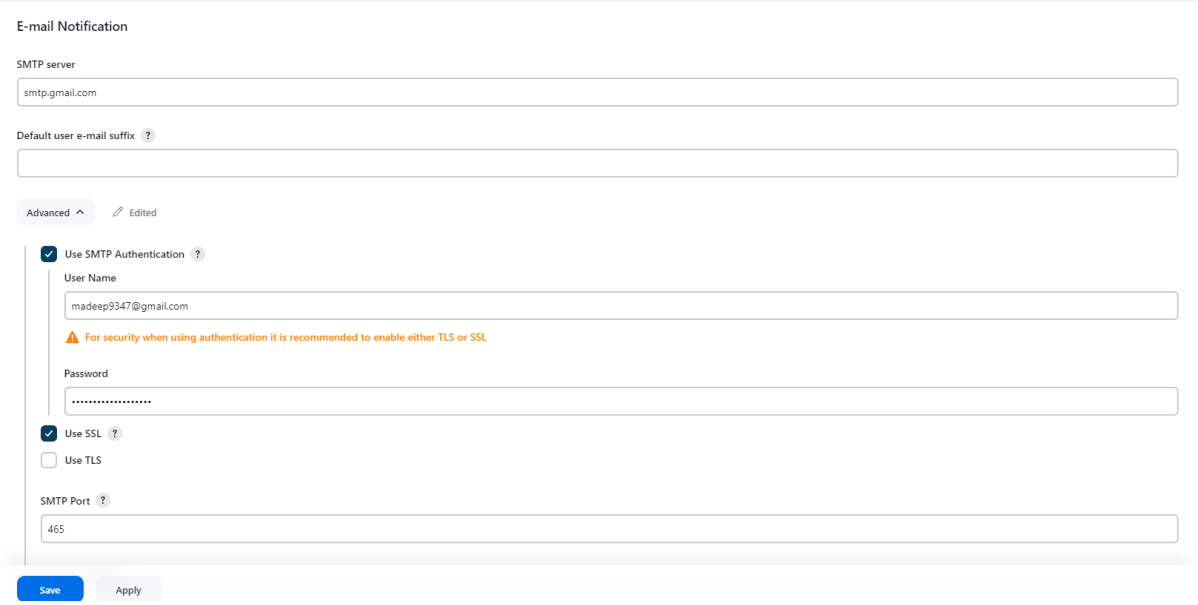


Email Notification Configurations:

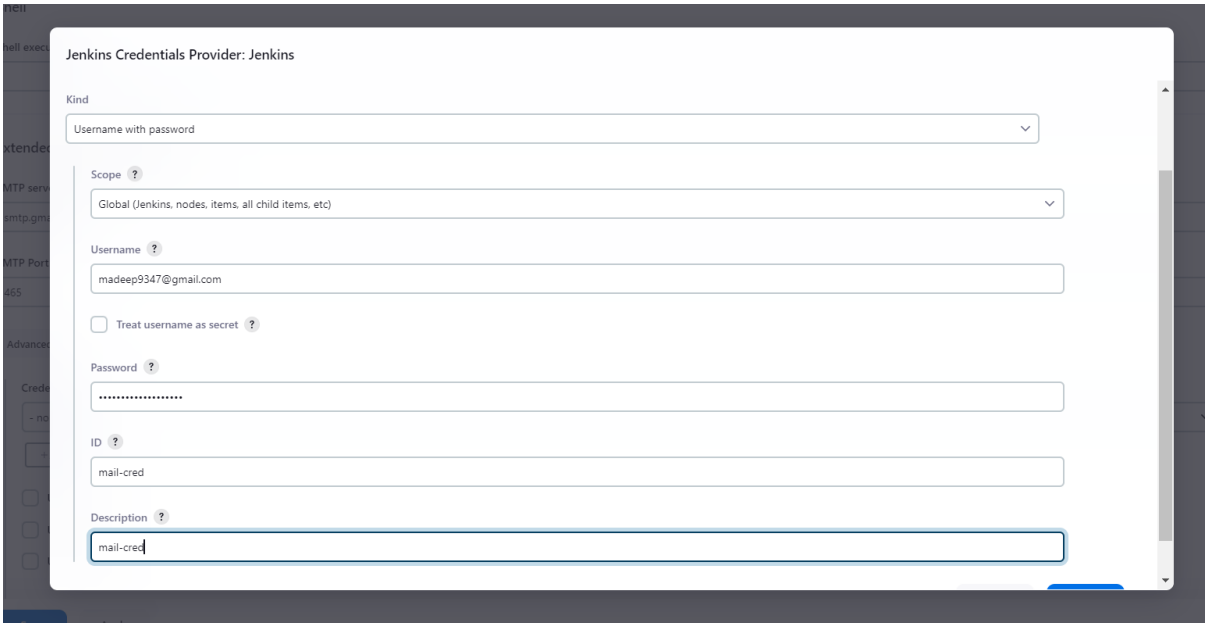
Goto this URL https://myaccount.google.com/apppasswords

generate apppassword and copy that password jijv akam wedd ujip

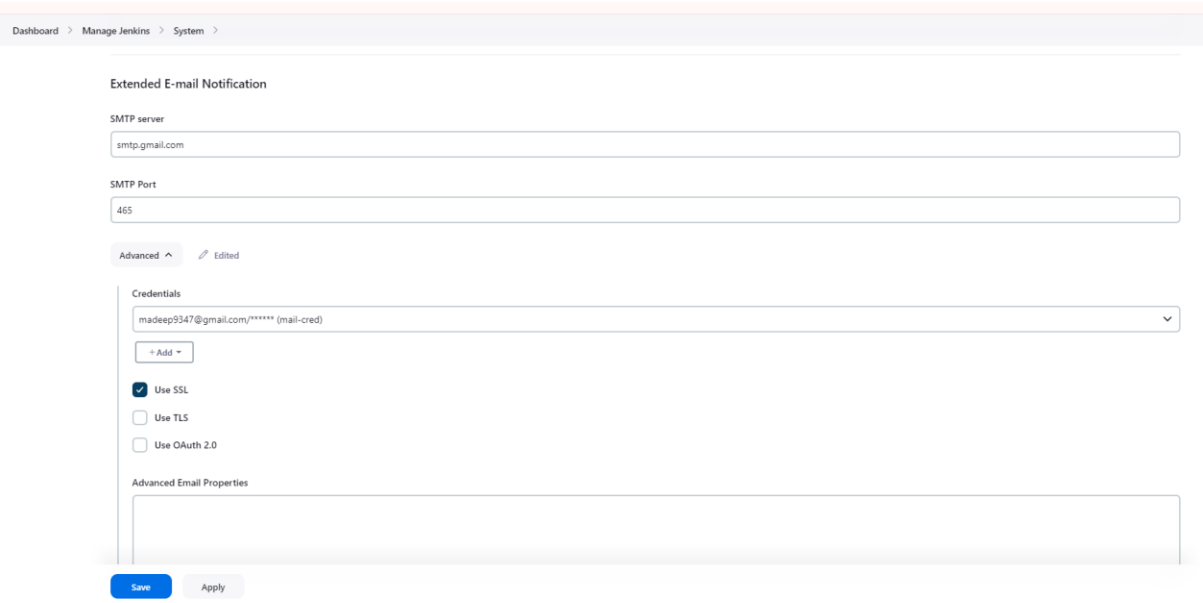
next go to Jenkins→manage Jenkins→system→E-mail Notification→smtp server= smtp.gmail.com, Advanced→ Use smtp Authentication→username="<yourgamilname>", password="<apppassword>", port= 465 and Test configuration by sending test e-mail.



Goto manage Jenkins→ credentials→ add the gmail and password



Now goto→ manage Jenkins→ system→ Extended E-mail Notification→ smtp-server=smtp.gmail.com, select the mail-cred



Now write the Jenkinsfile

```
pipeline {
  agent any

  tools {
    jdk 'jdk17'
    maven 'maven3'
  }

  environment {
    SCANNER_HOME = tool 'sonar-scanner'
  }

  stages {
    stage('Git Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/Madeep9347/Task-Master-Pro.git'
      }
    }

    stage('Compile') {
      steps {
        sh "mvn compile"
      }
    }

    stage('Test') {
      steps {
        sh "mvn test"
      }
    }

    stage('Trivy Scan File System') {
      steps {
        sh "trivy fs --format table -o trivy-fs-report.html ."
      }
    }

    stage('SonarQube Analysis') {
      steps {
        withSonarQubeEnv('sonar') {
          sh '''$SCANNER_HOME/bin/sonar-scanner \
            -Dsonar.projectKey=Mission \
            -Dsonar.projectName=Mission \
            -Dsonar.java.binaries=.'''
        }
      }
    }

    stage('Build') {
```

```
        steps {
            sh "mvn package -DskipTests=true"
        }
    }

    stage('Deploy Artifacts To Nexus') {
        steps {
            withMaven(globalMavenSettingsConfig: 'maven-setting', jdk: 'jdk17', maven: 'maven3') {
                sh "mvn deploy -DskipTests=true"
            }
        }
    }

    stage('Build & Tag Docker Image') {
        steps {
            script {
                withDockerRegistry(credentialsId: 'docker-cred') {
                    sh "docker build -t madeep2669/task-master-pro:latest ."
                }
            }
        }
    }

    stage('Trivy Scan Image') {
        steps {
            sh "trivy image --format table -o trivy-image-report.html madeep2669/task-master-
pro:latest"
        }
    }

    stage('Publish Docker Image') {
        steps {
            script {
                withDockerRegistry(credentialsId: 'docker-cred') {
                    sh "docker push madeep2669/task-master-pro:latest"
                }
            }
        }
    }
    stage ('deploy to kubernetes') {
        steps {
            withKubeConfig(caCertificate: '', clusterName: ' my-eks22', contextName: '', credentialsId:
'k8-token', namespace: 'webapps', restrictKubeConfigAccess: false, serverUrl:
'https://4D7D32432BFB24AA2106BBE7CF5CF5C1.gr7.ap-south-1.eks.amazonaws.com') {
                sh 'kubectl apply -f deployment-service.yml'
                sleep 60

            }
        }
    }
    stage ('verify the deployment') {
```

```
        steps {
            withKubeConfig(caCertificate: '', clusterName: ' my-eks22', contextName: '', credentialsId:
'k8-token', namespace: 'webapps', restrictKubeConfigAccess: false, serverUrl:
'https://4D7D32432BFB24AA2106BBE7CF5CF5C1.gr7.ap-south-1.eks.amazonaws.com') {
                sh 'kubectl get pods -n webapps'
                sh 'kubectl get svc -n webapps'
            }
        }
    }
  }
   post {
    always {
        script {
            def jobName = env.JOB_NAME
            def buildNumber = env.BUILD_NUMBER
            def pipelineStatus = currentBuild.result ?: 'UNKNOWN'
            def bannerColor = pipelineStatus.toUpperCase() == 'SUCCESS' ? 'green' : 'red'

            def body = """
                <html>
                <body>
                <div style="border: 4px solid ${bannerColor}; padding: 10px;">
                <h2>${jobName} - Build ${buildNumber}</h2>
                <div style="background-color: ${bannerColor}; padding: 10px;">
                <h3 style="color: white;">Pipeline Status: ${pipelineStatus.toUpperCase()}</h3>
                </div>
                <p>Check the <a href="${BUILD_URL}">console output</a>.</p>
                </div>
                </body>
                </html>
            """

            emailext (
                subject: "${jobName} - Build ${buildNumber} - ${pipelineStatus.toUpperCase()}",
                body: body,
                to: 'madeep9347@gmail.com',
                from: 'jenkins@example.com',
                replyTo: 'jenkins@example.com',
                mimeType: 'text/html',
                attachmentsPattern: 'trivy-image-report.html'
            )
        }
    }
  }
}
```

Not secure | 13.201.94.71:8080/job/Task-Master-pro/6/console

Dashboard > Task-Master-pro > #6

```
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withKubeConfig
[Pipeline] {
[Pipeline] sh
+ kubectl get pods -n webapps
NAME                                  READY   STATUS    RESTARTS   AGE
taskmaster-deployment-66dbb76c64-66gf7   1/1     Running   0          27m
taskmaster-deployment-66dbb76c64-8zf8z   1/1     Running   0          27m
[Pipeline] sh
+ kubectl get svc -n webapps
NAME             TYPE           CLUSTER-IP      EXTERNAL-IP                                                              PORT(S)        AGE
taskmaster-ssvc  LoadBalancer   10.100.62.149   a380b6908ef464567bebe17c654a8cca-1351288130.ap-south-1.elb.amazonaws.com   80:32331/TCP   27m
[Pipeline] }
[kubernetes-cli] kubectl configuration cleaned up
[Pipeline] // withKubeConfig
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] script
[Pipeline] {
[Pipeline] emailext
Sending email to: madeep9347@gmail.com
[Pipeline] }
[Pipeline] // script
```

hub.docker.com/repository/docker/madeep2669/task-master-pro/general

dockerhub | Explore | Repositories | Organizations | Search Docker Hub ctrl+K

madeep2669 / Repositories / task-master-pro / General

Using 0 of 1 private repositories.

General | Tags | Builds | Collaborators | Webhooks | Settings

madeep2669/task-master-pro

Updated 10 minutes ago

This repository does not have a description INCOMPLETE

This repository does not have a category INCOMPLETE

Docker commands

Public View

To push a new tag to this repository:

```
docker push madeep2669/task-master-pro:tagname
```

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	an hour ago	10 minutes ago

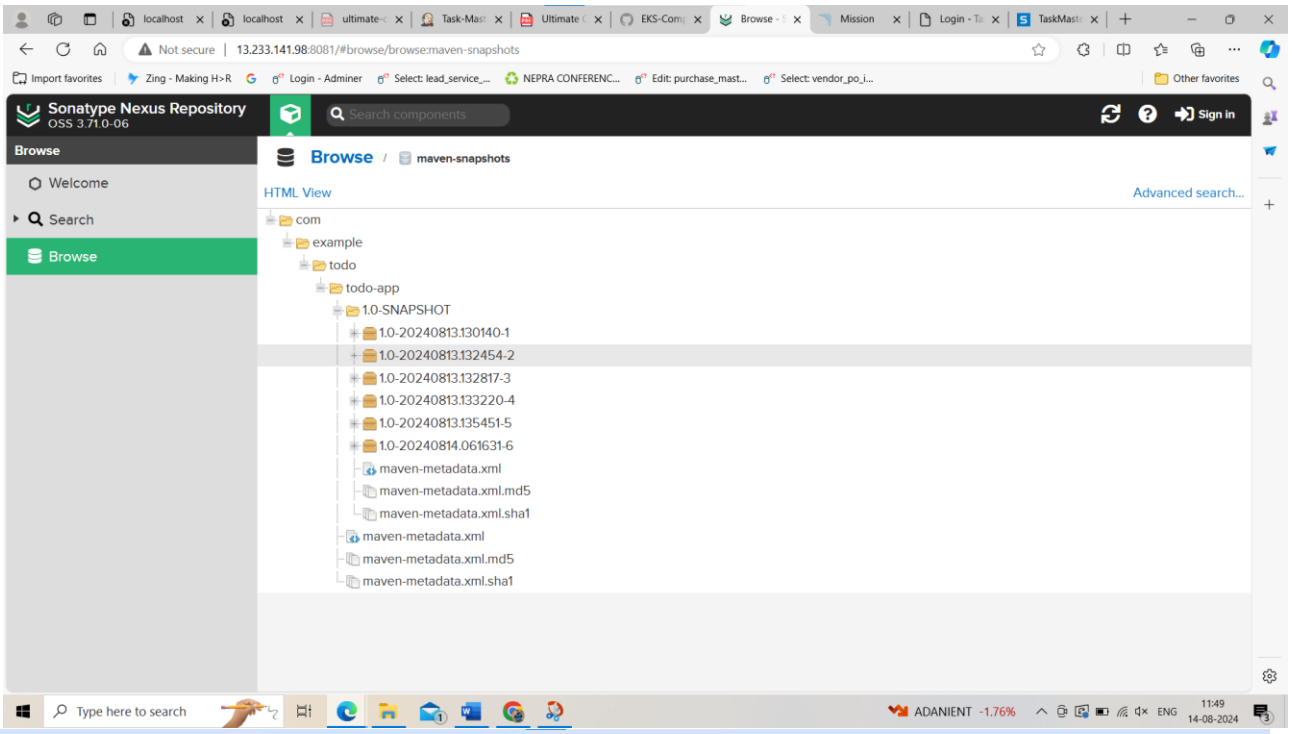
See all

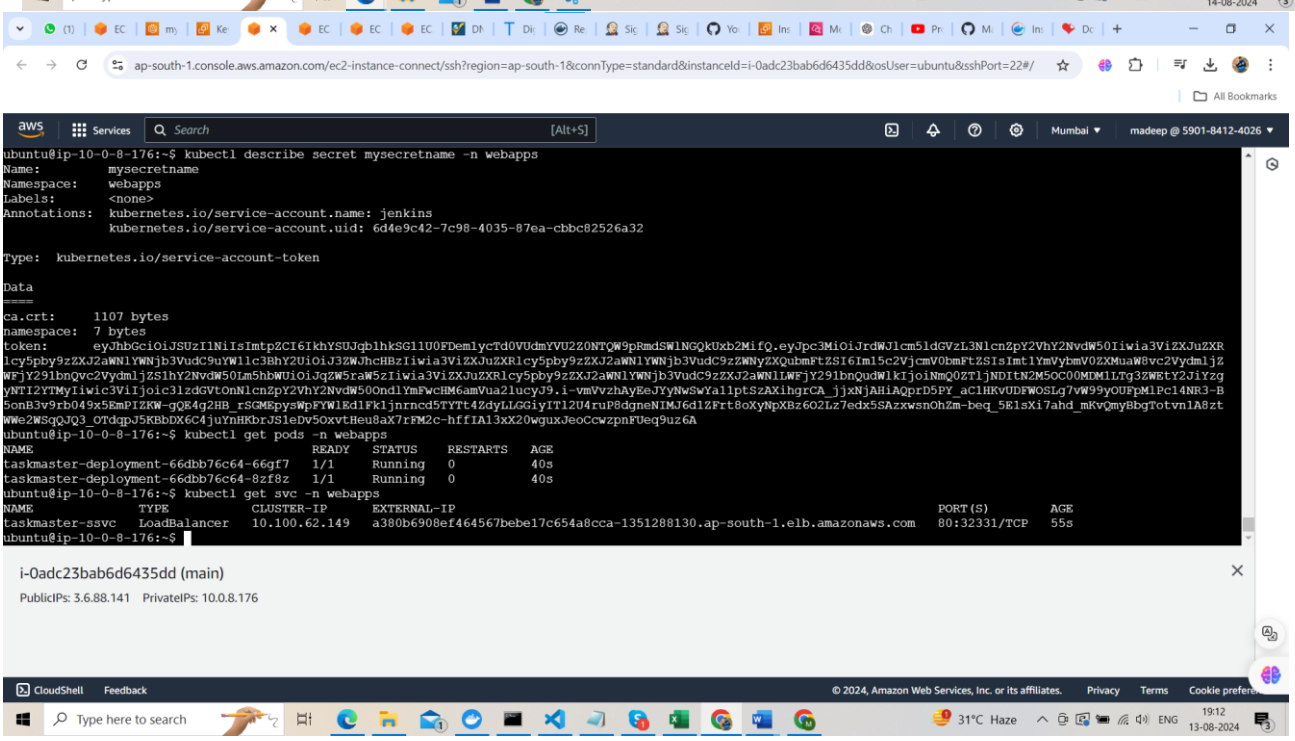
Automated Builds

Manually pushing images to Hub? Connect your account to GitHub or Bitbucket to automatically build and tag new images whenever your code is updated, so you can focus your time on creating.

Available with Pro, Team and Business subscriptions. Read more about automated builds.

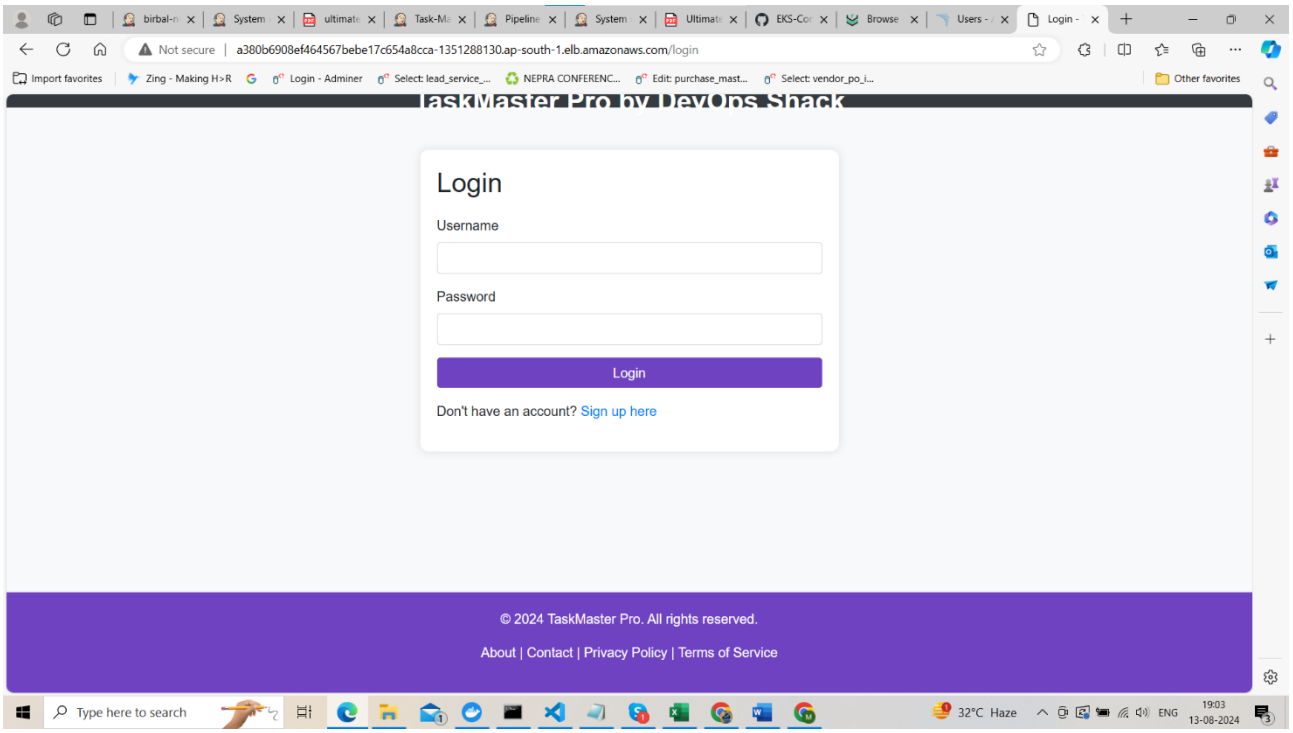
Upgrade

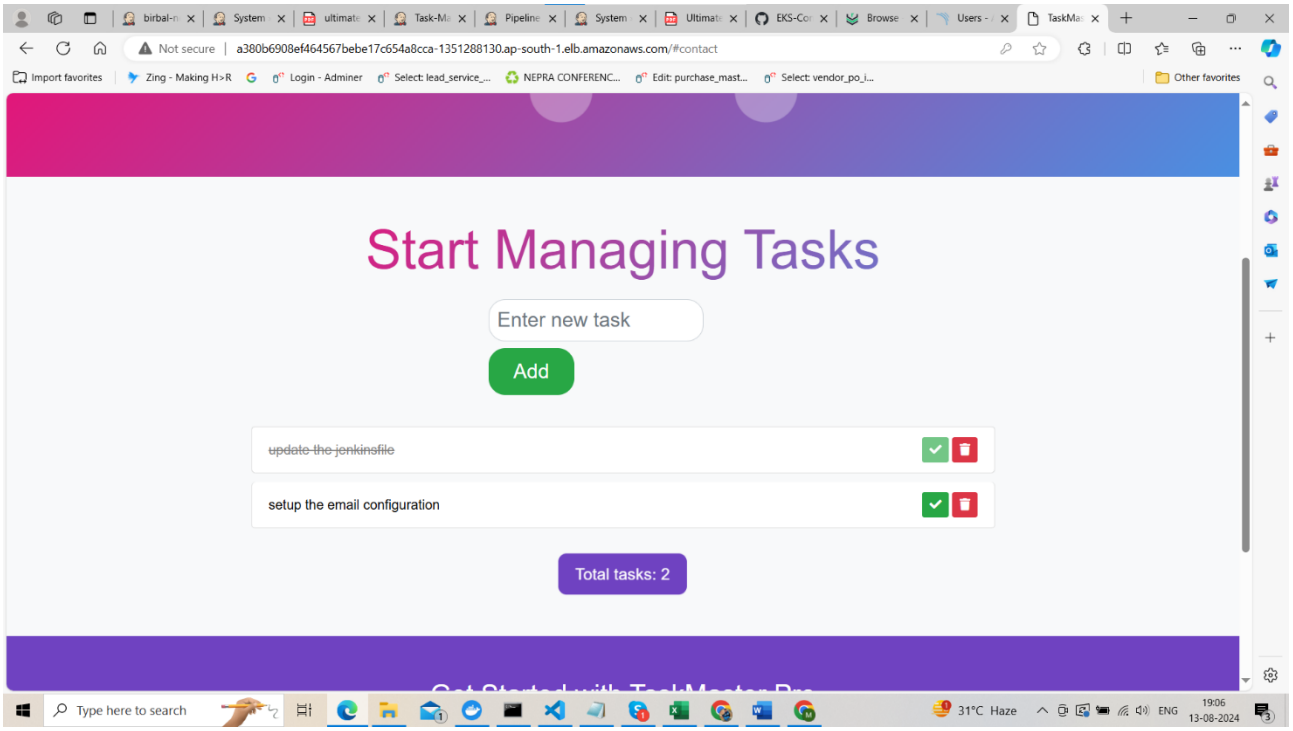




Access the Application using the External-ip

http://a380b6908ef464567bebe17c654a8cca-1351288130.ap-south-1.elb.amazonaws.com

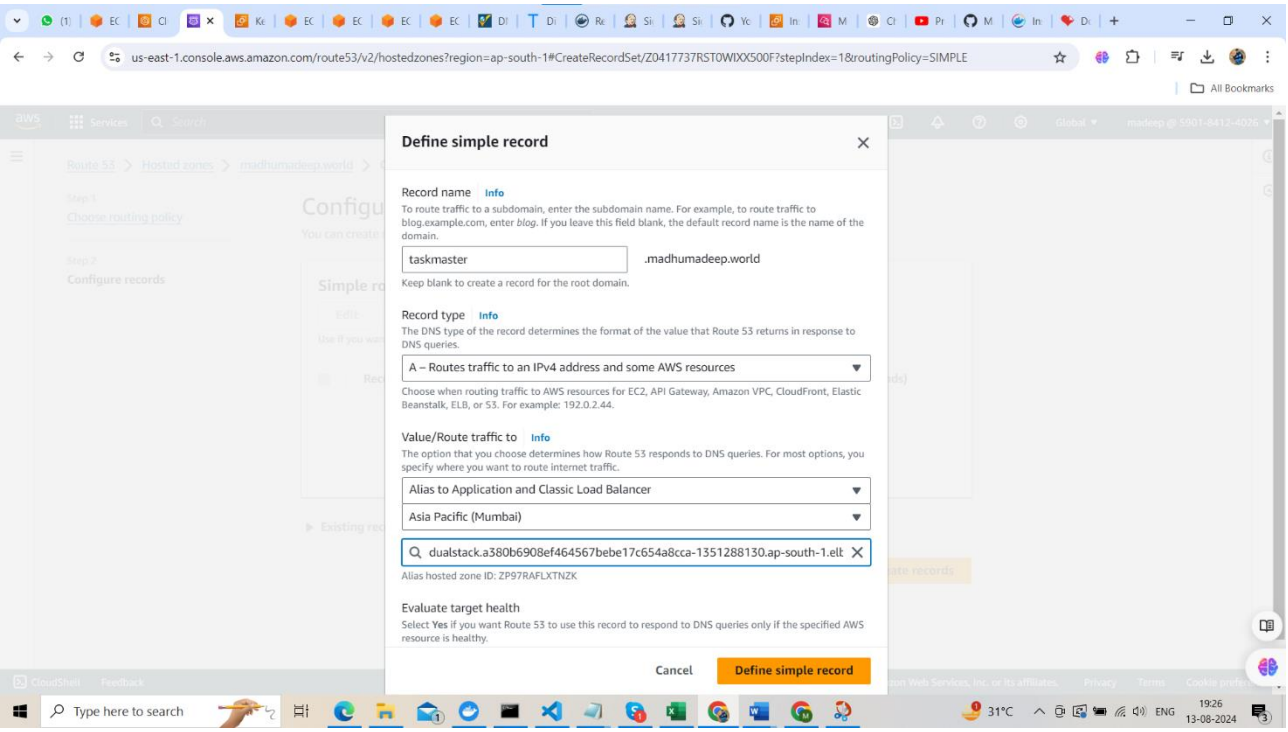

Not secure | a380b6908ef464567bebe17c654a8cca-1351288130.ap-south-1.elb.amazonaws.com/login
TaskMaster Pro by DevOps Shack
Login
Username
Password
Login
Don't have an account? Sign up here
© 2024 TaskMaster Pro. All rights reserved.
About | Contact | Privacy Policy | Terms of Service

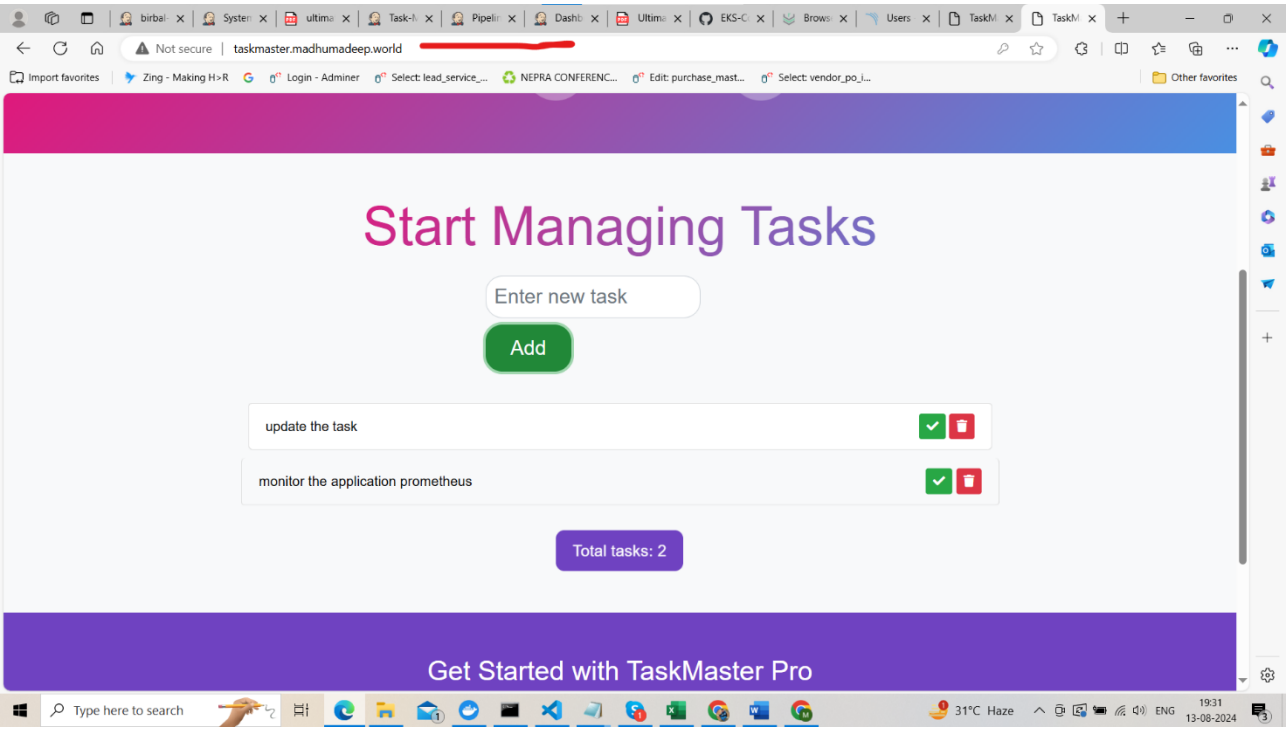

Not secure | a380b6908ef464567bebe17c654a8cca-1351288130.ap-south-1.elb.amazonaws.com/#contact
Start Managing Tasks
Enter new task
Add
update the jenkinsfile
setup the email configuration
Total tasks: 2

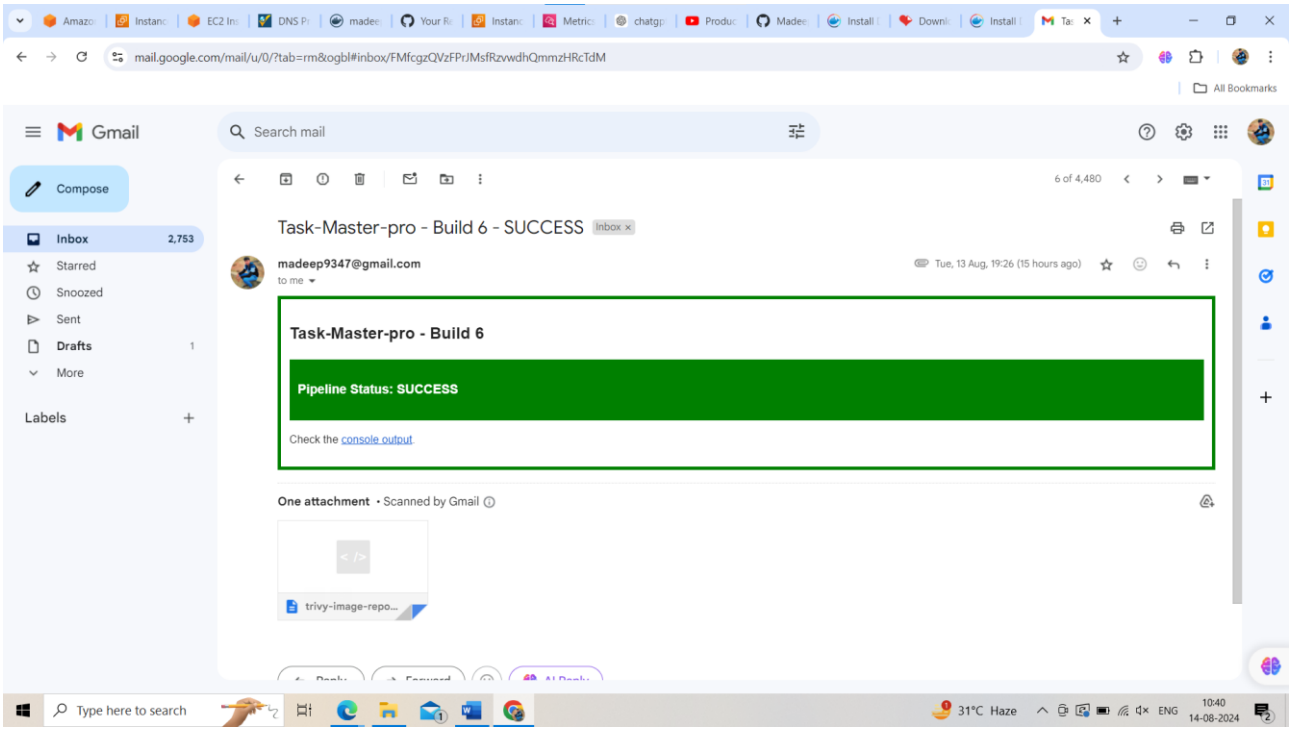
Map to Domain Name:

Goto → AWS Route 53

1 .Create simple record for load balancer






mail.google.com/mail/u/0/?tab=rm&ogbl#inbox/FMfcgzQVzFPrJMsfRzvwdhQmmzHRcTdM
Gmail
Search mail
Compose
Inbox 2,753
Starred
Snoozed
Sent
Drafts 1
More
Labels
6 of 4,480
Task-Master-pro - Build 6 - SUCCESS
Inbox
madeep9347@gmail.com
to me
Tue, 13 Aug, 19:26 (15 hours ago)
Task-Master-pro - Build 6
Pipeline Status: SUCCESS
Check the console output.
One attachment • Scanned by Gmail
trivy-image-repo...
Type here to search
31°C Haze
ENG
10:40
14-08-2024